Missing values and results — answers

Answers to the 16 items in `missing value and result.pdf`, prepared 2026-10-05, updated 2026-11-08 with values from the training PC.

> **Read this first.** Items 7, 8, 9, 15 and 16 were answered on the evaluation laptop (RTX 5060). On 2026-11-08 `collect_training_machine_facts.sh` was run on the PC that holds the original training outputs (user `MustafaJK`, WSL2). That run, plus the optimizer counters stored in the final checkpoints and the files' NTFS creation times, closed every remaining item (1–6, 10–14). All 16 items are now answered. **Correction:** the training PC's GPU is an **NVIDIA GeForce RTX 4060 8 GB**, not an RTX 3050 as assumed earlier. Raw files: `reports/paper/results/training_machine/`.

Status at a glance

#	Item	Status
1	Per-epoch training history	Done — all five models, from the original `progress.json`
2	Model A epoch timing	Done — measured
3	Model A generator-update count	Done — 2 per step (epochs 1–11), 3 (epochs 12–50)
4	Batch size for F and G	Done — 32, confirmed by checkpoint update counts
5	Landmark regressor validation error	Done — original weights
6	Regressor training time	Done — at most 48 s originally; 42.9 s re-timed on the same GPU
7	Native pixel size	Done
8	Per-class image counts	Done
9	Qualitative visual description	Done
10–12	CPU, RAM, CUDA, cuDNN, driver, WSL, Windows	Done — training PC
13	Frozen dependency lists	Done — training PC
14	MediaPipe version, dataset source and date	Done — MediaPipe 0.10.14, parquet dated 2026-07-06
15	Fixed-seed evaluation pass	Done (the laptop, seed 42)

16	Full per-class accuracy, all five models	Done

Files and numbers from the run

1. Per-epoch training history done

The original `progress.json` of all five models (50 epochs each, with per-epoch losses and `epoch_seconds`) is now committed under `reports/paper/results/training_machine/cgan_{A_128,B_128mp,C_128struct,F_128fusion,G_128plus}/`, together with the original `loss_history.csv` for A, B and C. F and G never wrote a `loss_history.csv` on the training PC; their per-epoch losses are in `progress.json` and in the log rebuilds `reports/paper/results/history_from_logs/loss_history_{F,G}.csv`. Model A's history, previously missing, is now available: `training_machine/cgan_A_128/loss_history.csv` (columns d, g_adv, g_pixel, g_total, gd_ratio, lam, epoch_seconds).

2. Model A epoch timing done

Measured from `epoch_seconds` in the original `progress.json`: **mean 517.5 s per epoch, range 399.3–567.5 s, total 25,873.2 s (7.19 h)** over 50 epochs. This agrees with the earlier estimate from `driver.log` (about 26,000 s). Epochs 1–11 take about 400 s and epochs 12–50 about 550 s, because the generator gets a third update per step from epoch 12 (item 3).

All five models, from their original `progress.json`:

Model	Mean s/epoch	Range (s)	Total s	Total h
A	517.5	399.3–567.5	25,873.2	7.19
B	538.8	417.4–574.4	26,941.4	7.48
C	232.4	224.5–261.9	11,618.5	3.23
F	239.2	235.7–261.9	11,962.5	3.32
G	289.5	283.2–308.4	14,475.1	4.02

G also spent 818 s training its auxiliary classifier before the GAN epochs.

3. Model A generator-update count done

2 generator updates per discriminator step in epochs 1–11, and 3 in epochs 12–50 (139 update-epochs in total, i.e. 22 + 117), the same schedule as Model B.

How it was derived: `src/train_model_a.py` starts at `G_UPDATES_BASE = 2` and, after each epoch, uses 3 updates if that epoch's G/D ratio exceeded `G_D_RATIO_MAX = 2.0` (4 if it exceeded 3.0). The original `gd_ratio` values rise from 0.80 to 1.69 over epochs 1–10, reach 2.19 at epoch 11, and stay between 2.16 and 2.20 until epoch 50, so 4 updates were never triggered. The measured epoch time confirms the switch: 401.5 s at epoch 11, then 548.9 s at epoch 12.

Direct count from the final checkpoint (`ckpt-50`): the generator optimizer applied **234,676** updates and the discriminator **84,409**, a ratio of 2.780, exactly the schedule's 139/50 = 2.780. With 1,689 steps per epoch (54,049 images, batch 32) the schedule predicts 234,771 generator updates; the 95 missing ones are steps skipped by mixed-precision loss scaling (inf/NaN gradients), which the optimizer does not count. File: `reports/paper/results/training_machine/optimizer_iterations.txt` .

4. Batch size for F and G done — 32

Direct proof from the checkpoints. `progress.json` has no steps-per-epoch field, but each final checkpoint stores how many updates the optimizer applied. All datasets drop the last partial batch, so at batch 32 a model on 48,645 training images makes 1,520 steps per epoch, i.e. 76,000 discriminator updates in 50 epochs:

Model	Training images	D updates in ckpt-50	Expected at batch 32	Skipped (loss scaling)
C (reference, batch 32)	48,645	75,971	76,000	29
F	48,645	75,969	76,000	31
G	48,645	75,970	76,000	30

Batch 16 would give about 152,000 updates and batch 64 about 38,000, so **F and G used batch 32 (1,520 steps per epoch)**. The 29–31 missing updates are steps skipped by mixed-precision loss scaling.

Supporting evidence: the training PC's shell history contains no `ARASL_BATCH` override, and `src/config.py` sets `BATCH_SIZE = int(os.environ.get("ARASL_BATCH", 32))` (`train_model_g.py` notes "Batch stays 32"). Together with F's epoch time being within 5% of C's on the same network (236 s against 226 s), everything agrees. The `761/761` steps in `train_G.log` belong to G's auxiliary classifier (batch 64), not the GAN.

5. Landmark regressor validation error done — original weights

On 2026-11-08 the **original** weights (`outputs/cgan_B_128mp/checkpoints/landmark_regressor.weights.h5` , written 2026-07-06 19:50:37, the weights Models B, F and G used) were evaluated on the training PC with `src/eval_landmark_regressor.py` , on the same split: 14,834 detected images (27.45%), 13,350 train, 1,484 validation.

Validation error	Original regressor	Constant baseline (training mean)	Reduction

MSE, normalised (all 63 values)	0.00283	0.00767	63%
MAE, normalised (all 63 values)	0.0305	0.0561	46%
MSE, pixels² (x, y only)	69.6	188.5	63%
MAE, pixels (x, y only)	5.82	10.74	46%
Mean keypoint distance, pixels	9.18	16.87	46%

These are the numbers to report in the manuscript. The reproduction below, made earlier on the laptop, is within 5% of them (mean distance 9.49 px against 9.18 px), which shows the training procedure is reproducible. File: `reports/paper/results/landmark_regressor_eval.json` ; log: `reports/paper/results/training_machine/landmark_regressor_eval.log` .

Earlier reproduction (kept for reference):

Before the original weights were available, the regressor was **reproduced** on the laptop on 2026-11-07 with the repository's own code: landmarks re-extracted with MediaPipe 0.10.14, then the regressor trained with `train_regressor()` from `src/train_model_b.py` (seed 42, same 20-epoch schedule and callbacks).

The reproduction matches the original pipeline closely: hand detection is **27.45%** (14,834 of 54,049 images), exactly the rate the training PC recorded. Validation set: the last 10% of the detected images (1,484 images; 13,350 for training), as Keras `validation_split` does in the original.

Validation error	Reproduced regressor	Constant baseline (training mean)	Reduction
MSE, normalised (all 63 values)	0.00295	0.00767	61%
MAE, normalised (all 63 values)	0.0318	0.0561	43%
MSE, pixels² (x, y only)	72.6	188.5	61%
MAE, pixels (x, y only)	6.03	10.74	44%
Mean keypoint distance, pixels	9.49	16.87	44%

- **Reading it:** on a 128 × 128 image, a predicted keypoint lands on average 9.5 px from the MediaPipe keypoint, against 16.9 px for always predicting the average hand. The regressor clearly learned hand-specific structure, but its error is far from zero, which limits how precise the landmark supervision for B, F and G can be.
- **Pixel units:** pixel errors use only x and y (scaled by 128); MediaPipe's z has no pixel meaning.
- **Consistency check:** the original B training log shows a landmark loss of about 0.003 on generated images, the same scale as this validation MSE.

- **Superseded:** these numbers come from a retrained regressor. Use the original-weights table above.

File: `reports/paper/results/landmark_regressor_eval_repro.json`; log: `reports/paper/logs/repro_landmark_regressor.log`.

6. Regressor training time done

Original run: at most 48 s on the RTX 4060. Windows keeps a creation time for each file. In Model B's first launch (`driver.log`: START B 19:46:59 on 2026-07-06), `checkpoints/classes.npy` was created at **19:49:49**, right after image loading, and `landmark_regressor.weights.h5` at **19:50:37**, written once after training. Everything in between (loading the cached landmarks, building the models and the 20 regressor epochs) took 48 s.

Re-timed on the same GPU: 42.9 s. The same `train_regressor()` was run again on the training PC on 2026-11-08 (new weights path, originals untouched): 20 epochs in 42.9 s, which fits inside the 48 s window. Suggested wording: "the regressor trains in under a minute (about 43 s) on the RTX 4060".

Files: `training_machine/extra.txt`, `training_machine/regressor_gpu_retime.log`, `landmark_regressor_eval_gpu_retime.json`. The re-trained copy's error (10.9 px) differs from the original's (9.18 px) because GPU training is not bit-deterministic; the manuscript should use the original weights' numbers from item 5.

Earlier evidence, kept for reference:

- **Reproduction on the laptop (CPU):** 560.9 s (9.3 min) for 20 epochs. The learning rate was halved at epochs 9 and 18, and the best epoch was 19.
- **Original on the training PC (GPU):** not recorded. Model B's final launch loaded the regressor already trained, so it was trained in one of two earlier short launches (319 s and 526 s, each including about 3 minutes of data loading). That puts it **under roughly 6 minutes**. The training PC confirms the timing: the original weights file was written at **2026-07-06 19:50:37**, 3.6 minutes after Model A finished on the same GPU (19:46:59), and Model B's real run started about 20:10.
- **Landmark extraction:** 3,000.6 s on the training PC; on the laptop the run was interrupted once; the resumed part took 2,442 s for the last 39,049 images, which scales to about 3,400 s for all 54,049.

7. Native pixel size done

The images are not all the same size. All are 8-bit grayscale PNG.

Size	Images	Where
64 × 64	53,401	all classes
256 × 256	638	kaaf (320), meem (318)
768 × 1024	10	haa

8. Per-class image counts done

Total 54,049 images in 32 classes. Largest class is `ain` (2,114), smallest is `yaa` (1,293): an imbalance ratio of 1.63. Mean 1,689, standard deviation 184, coefficient of variation 0.109.

Class	n	Class	n	Class	n	Class	n
ain	2114	gaaf	1705	laam	1832	taa	1838
al	1343	ghain	1977	meem	1765	thaa	1766
aleff	1672	ha	1592	nun	1819	thal	1582
bb	1791	haa	1526	ra	1659	toot	1791
dal	1634	jeem	1552	saad	1895	waw	1371
dha	1723	kaaf	1774	seen	1638	ya	1722
dhad	1670	khaa	1607	sheen	1507	yaa	1293
fa	1955	la	1746	ta	1816	zay	1374

9. Qualitative description of the visual differences done

From viewing `real_vs_generated.png` and the five `samples_*.png` grids:

- **Models A and B (noise only).** Outputs are hand-like, with recognisable pose for open-hand letters such as *al*, *bb*, *la* and *sheen*. Fingers are soft and often merged or missing. Closed-fist letters (*fa*, *gaaf*, *ha*, *khaa*) collapse into fragmentary dark masses. Backgrounds are uniformly light. Pose varies widely between samples of the same letter. B shows no visible improvement over A.
- **Model C.** Outputs follow the reference hand's silhouette with sharp contours. Periodic checkerboard artefacts appear in the background: scattered patches on light backgrounds, and across the whole frame on dark or textured ones (*dha*, *ghain*, *ha*, *khaa*, *laam*, *thal*).
- **Model F.** Same structure as C. The artefacts are largely gone on light backgrounds but remain on dark or textured ones.
- **Model G.** The cleanest of the five. Only faint residual speckle remains on a few letters (*ha*, *khaa*, *laam*, *thal*, *aleff*).
- **C, F and G together.** The eight samples per letter are visually near-identical. The noise vector has little visible effect, so variety comes from the structure map, not from the noise.

Environment and software

10–12. Hardware and versions done

Recorded on the training PC on 2026-11-08 (`reports/paper/results/training_machine/environment.txt`). The laptop used for items 15–16 is shown for comparison.

	Training PC (models A–G)	**Evaluation laptop**
CPU	Intel Core i7-12700K	Intel Core i7-14650HX
RAM	32 GB physical; 17 GB available to WSL2	15.7 GB
GPU / driver	**NVIDIA GeForce RTX 4060, 8 GB**, driver 616.92	RTX 5060 Laptop 8 GB, driver 610.62
Windows	build 10.0.26200.9711	11 Pro, build 26200
WSL	3.0.3.0, Ubuntu 26.04 LTS, kernel 6.18.54.1	2.7.10.0, Ubuntu 26.04 LTS, kernel 6.18.33.2
Python / TensorFlow / Keras	3.11.15 / 2.21.0 / 3.15.0	3.11.15 / 2.21.0 / 3.15.0
TensorFlow build versions	CUDA 12.5.1, cuDNN 9	CUDA 12.5.1, cuDNN 9
CUDA / cuDNN wheels loaded	CUDA runtime 12.9.79, cuBLAS 12.9.2.10, cuDNN 9.24.0.43	CUDA 12.9.x, cuDNN 9.24.0.43

- **GPU correction:** the manuscript (and earlier versions of this page) should say RTX 4060 8 GB, not RTX 3050.
- **CUDA line:** "CUDA 12.5.1, cuDNN 9" is TensorFlow's build-time string. The libraries actually loaded are CUDA 12.9 and cuDNN 9.24. Suggested wording: "TensorFlow 2.21 (built against CUDA 12.5 / cuDNN 9) with CUDA 12.9 and cuDNN 9.24 runtime libraries".
- **RAM:** the PC has 32 GB, but WSL2 only gets about 17 GB by default. That explains the "avoids OOM at 16 GB" comment in `src/train_model_b.py` ; both statements are true.
- `nvcc` is not installed on the training PC; CUDA comes only from the pip `nvidia-*` wheels.

13. Frozen dependency lists done

The training PC's lists are committed as `requirements_tf.txt` (TensorFlow env `arasl` , protobuf 7.35.1) and `requirements_mp.txt` (MediaPipe env `mp` , protobuf 4.25.9) at the repository root. They match the laptop's lists in `reports/paper/results/seed42/` on the key packages, and the protobuf split matches the "4.x vs 7.x" note in the code.

14. MediaPipe version and dataset source done

- **MediaPipe:** 0.10.14 on the training PC (protobuf 4.25.9), the same as on the laptop.
- **Dataset:** `pain/ArASL_Database_Grayscale` on Hugging Face. On the training PC `data/arasl.parquet` is dated **2026-07-06 09:34:58 (+03:00)**, and the extracted image folder's `_counts.json` 10:00:16 the same morning, three hours before Model A started (12:30). The Hugging Face cache holds no copy of the dataset, so the file timestamp is the best record of the download date.

Additional runs

15. Fixed-seed evaluation pass done

Run on the laptop (CPU) on 2026-11-07 with seed 42. One reference classifier was trained, saved and used by *both* `paper_eval.py` and `confusion_matrix.py`. Both logs print the same provenance line:

```
path=reports/paper/results/seed42/reference_classifier_seed42.keras
mtime=2026-11-07 18:06:09   seed=42   samples_per_class=40 (paper_eval) / 60 (confusion_matrix)
reference classifier on real held-out = 0.9754
```

Model	Recognition	Diversity	SSIM	Held-out recognition	Generalisation gap	Confusion-matrix accuracy
A	0.6211	0.1779	—	—	—	0.6323
B	0.6484	0.1926	—	—	—	0.6594
C	0.7172	0.4134	0.7521	0.6969	0.0203	0.7224
F	0.8609	0.3722	0.8249	0.8370	0.0240	0.8656
G	0.9383	0.4010	0.8251	0.9062	0.0321	0.9380

- **Why two recognition columns:** both come from the same classifier, but `paper_eval.py` scores 40 samples per class and `confusion_matrix.py` scores 60, with different noise seeds. They differ by less than 0.012.
- **New numbers replace the old:** they are slightly lower than the earlier unseeded logs (for example G 0.9383 against 0.9461), because the classifier is a different instance. The ranking A < B < C < F < G is unchanged. The manuscript's headline table and Table 17 should both take their values from this run.
- **Consistency check:** diversity for A, B, F and G matches the original run to four decimals, so the generators behave identically on the laptop.
- **Reproducibility note:** training was interrupted by low memory and resumed from saved epoch 11 (re-seeded with 42 + 11). Re-running gives the same result, but it is not bit-identical to an uninterrupted run.

Files: `reports/paper/results/seed42/metrics.json`, `metrics.csv`, `confusion_[A-G].png`, `confusion_summary.json`; logs in `reports/paper/logs/*_seed42.log`.

16. Full per-class accuracy, all five models done

Fraction of the 60 generated images per letter that the reference classifier assigned to the intended letter (from `reports/paper/results/seed42/per_class_accuracy.csv`).

Class	A	B	C	F	G
ain	0.833	0.800	0.667	0.850	0.833
al	0.967	0.967	0.967	0.967	0.983
aleff	0.417	0.367	0.333	0.650	0.833
bb	0.817	0.783	0.517	0.817	0.883
dal	0.633	0.383	0.883	0.883	0.933
dha	0.717	0.583	0.667	0.833	0.933
dhad	0.817	0.850	0.767	0.833	0.933
fa	0.450	0.683	0.483	0.783	0.883
gaaf	0.100	0.300	0.533	0.833	0.933
ghain	0.333	0.467	0.700	0.883	0.983
ha	0.733	0.383	0.400	0.567	0.933
haa	0.700	0.567	0.767	0.883	0.933
jeem	0.250	0.500	0.783	0.900	0.967
kaaf	0.483	0.617	0.617	0.783	0.917
khaa	0.500	0.617	0.850	0.950	1.000
la	0.767	0.883	0.833	0.867	0.967
laam	0.767	0.750	0.817	0.833	0.883
meem	0.817	0.800	0.650	0.850	0.950
nun	0.817	0.850	0.867	0.983	0.983
ra	0.617	0.633	0.633	0.833	0.917
saad	0.650	0.717	0.633	0.883	0.967
seen	0.850	0.800	0.883	0.983	1.000

sheen	0.900	0.933	0.983	1.000	1.000
ta	0.800	0.867	0.900	0.950	1.000
taa	0.700	0.600	0.633	0.833	0.917
thaa	0.167	0.433	0.583	0.833	0.933
thal	0.450	0.517	0.833	0.933	1.000
toot	0.600	0.733	0.750	0.917	0.967
waw	0.533	0.633	0.767	0.917	0.917
ya	0.867	0.700	0.783	0.933	0.900
yaa	0.950	0.817	0.983	1.000	1.000
zay	0.233	0.567	0.650	0.733	0.833
mean	**0.632**	**0.659**	**0.722**	**0.866**	**0.938**

Three largest confusions per model (intended → predicted, share of that letter's samples):

Model	Top confusions
A	gaaf → fa (0.483); thaa → taa (0.433); ghain → ain (0.383)
B	gaaf → ha (0.300); ha → fa (0.267); haa → khaa (0.250)
C	kaaf → seen (0.200); aleff → khaa (0.183); saad → ya (0.183)
F	kaaf → seen (0.217); ha → kaaf (0.100); ra → toot (0.100)
G	aleff → nun (0.067); kaaf → seen (0.067); aleff → ghain (0.050)

Training PC run (2026-11-08)

```
bash collect_training_machine_facts.sh
```

Run on the training PC; results committed in `c402427` on `reviewer-missing-values`. It copied the five histories, summarised every model's timing (`progress_summary.txt`), evaluated the original regressor, recorded the environment and wrote both `pip freeze` files. The only missing inputs were F's and G's `loss_history.csv`, which were never written there. Extra evidence for items 4, 6 and 14 is in `training_machine/extra.txt`; optimizer update counts for items 3 and 4 are in `training_machine/optimizer_iterations.txt`.

Files added or changed

- `src/paper_eval.py`, `src/confusion_matrix.py` — seeded, shared classifier, full per-class output
- `src/eval_landmark_regressor.py` — regressor validation error and baseline
- `src/paper_charts_from_logs.py` — F and G loss charts
- `collect_training_machine_facts.sh` — one-shot collector for the training PC
- `reports/paper/results/history_from_logs/` — four loss histories and `timing_summary.json`
- `reports/paper/charts/loss_model_F.png`, `loss_model_G.png`
- `reports/paper/results/seed42/` — seeded evaluation results, per-class accuracy, confusion matrices, saved classifier, the laptop's dependency lists
- `src/data.py`, `src/train_model_c.py` — image loader writes into one preallocated array (same values, about half the peak memory)
- `run_missing_values.sh` — resumable run for items 5, 6, 15 and 16
- `experiments/scripts/extract_lm_full.py` — reads images memory-mapped (same values, far less memory)
- `reports/paper/results/landmark_regressor_eval_repro.json` — reproduced regressor error and baseline
- `reports/paper/results/training_machine/` — original histories, timing summary, environment, file times, extra evidence (training PC)
- `reports/paper/results/landmark_regressor_eval.json` — original regressor error and baseline (training PC)
- `requirements_tf.txt`, `requirements_mp.txt` — training PC dependency lists